PROJECT DOCUMENTATION

AC Service Booking Website

An Online Platform for Booking Air Conditioner Services

Prepared by: Sharoz

BCA (Cybersecurity), KCMT, Bareilly

Technology Stack: Node.js · Express.js · MongoDB

Table of Contents

1. Introduction

The AC Service Booking Website is a full-stack web application designed to simplify the process of booking air conditioner installation, repair, and maintenance services online. Inspired by on-demand service platforms such as Urban Company, the system connects customers with service providers through a clean, easy-to-use interface, removing the friction of phone-based bookings and manual scheduling.

The application is built using the MEN stack (MongoDB, Express.js, Node.js) on the backend, paired with a responsive HTML, CSS, and JavaScript frontend. It allows users to browse available AC services, select a convenient time slot, submit a booking request, and track the status of their service — all from a single platform.

2. Project Objectives

- Provide a simple, user-friendly interface for browsing and booking AC services.
- Automate service scheduling and reduce dependency on manual, phone-based booking.
- Build a reliable backend using Express.js and MongoDB to manage services, users, and bookings.
- Ensure the platform is responsive and accessible across desktop and mobile devices.
- Design a modular, maintainable codebase that supports future feature additions.

3. Scope of the Project

The current version of the application focuses on the core booking workflow — service discovery, appointment scheduling, and request management. It is designed with a modular architecture so that features such as online payments, real-time technician tracking, customer reviews, and an admin analytics dashboard can be added in future iterations without major rework of the existing codebase.

4. Technology Stack

Layer	Technology Used	Purpose
Frontend	HTML, CSS, JavaScript	User interface, layout, and client-side interactivity
Backend	Node.js, Express.js	Server-side logic, routing, and REST API endpoints
Database	MongoDB	Storing user, service, and booking data
Templating/Views	Express Views (EJS/HTML)	Rendering dynamic pages served by the backend

5. System Architecture

The application follows the standard Model-View-Controller (MVC) architectural pattern, which separates data handling, business logic, and presentation for easier maintenance and scalability:

Component	Responsibility
Models	Define schemas for Users, Services, and Bookings, and handle data validation via Mongoose

Controllers	Contain the business logic that processes requests and prepares responses
Routes	Map incoming HTTP requests to the appropriate controller functions
Views / Public	Render the frontend pages and serve static assets (CSS, JS, images)

6. Project Structure

The codebase is organized into clearly separated folders to keep the project modular and easy to navigate:

- routes/ — API endpoint definitions (e.g., service listing, booking creation, status updates)
- models/ — Mongoose schemas for User, Service, and Booking collections
- controllers/ — Request handling logic for each route
- public/ — Static assets such as CSS, client-side JavaScript, and images
- views/ — Frontend page templates rendered to the user
- documentation/ — Project notes and reference material

7. Core Features

- User-friendly, intuitive interface for browsing AC services
- Online AC service booking with date and time slot selection
- Fully responsive design that works across screen sizes
- Persistent data storage and retrieval using MongoDB
- RESTful backend APIs built with Express.js for all core operations

8. Implementation

The implementation was carried out in the following stages:

1. UI Design — Designed the frontend pages for service browsing, booking forms, and confirmation screens with a focus on clarity and ease of use.
2. Backend Setup — Built the Express.js server, configured middleware, and structured the application using the MVC pattern.
3. Database Integration — Connected the application to MongoDB and defined schemas for users, services, and bookings.
4. Models and Routes — Created Mongoose models and corresponding Express routes/controllers to handle CRUD operations for services and bookings.
5. Booking Workflow Testing — Verified the end-to-end flow from service selection to booking confirmation.
6. Bug Fixes and Optimization — Resolved identified issues and optimized queries and page load performance.

Together, these stages moved the project from initial architecture planning through to a working, testable booking application, following a modular structure intended to simplify future maintenance and feature additions.

9. Planned Features & Pages

Beyond the core booking workflow, the following pages and capabilities are planned to expand the platform into a more complete, production-ready service:

9.1 User-Side Pages

- Landing / Home page — service overview, hero banner, testimonials
- Service listing page — AC installation, repair, gas refill, general service, with price ranges
- Service detail page — description, pricing, and a Book Now action
- Booking form — date/time slot picker, address, and contact details
- Booking confirmation page — booking summary and booking ID
- My Bookings / order history — view past and upcoming bookings after login
- Login / Signup — email or phone-based authentication

9.2 Admin-Side Pages

- Admin dashboard — total, pending, and completed bookings at a glance
- Manage services — add, edit, or remove service types and pricing
- Manage bookings — update status (pending → assigned → completed)
- Technician assignment — assign a booking to a specific technician

9.3 Nice-to-Have Enhancements

- Real-time status tracking (e.g., Socket.io-based "technician on the way" updates)
- Payment gateway integration (Razorpay/Stripe test mode)
- Email/SMS notifications on booking confirmation (Nodemailer/Twilio)
- Ratings and reviews after service completion
- Search and filter services by category or price

9.4 Security Enhancements

Given the project's alignment with a cybersecurity focus, the following hardening measures are planned to strengthen the application beyond baseline functionality:

- Input validation and sanitization to prevent NoSQL injection against MongoDB
- JWT-based authentication with bcrypt password hashing
- Rate limiting on booking and login APIs to mitigate brute-force attempts
- HTTPS enforcement and secure HTTP headers via helmet.js
- Basic role-based access control (RBAC) to separate user and admin privileges

10. Testing

Manual testing was performed on the core booking workflow to confirm that services display correctly, bookings are saved to the database, and confirmation feedback is shown to the user. Edge cases such as empty form submissions and invalid inputs were checked to ensure basic validation behaves as expected.

11. Future Enhancements

- Integration of an online payment gateway for service charges
- User authentication and login-based booking history
- Admin dashboard for managing services, technicians, and bookings
- Real-time booking status updates and notifications
- Customer ratings and reviews for completed services

12. Conclusion

The AC Service Booking Website successfully demonstrates a working full-stack booking platform built on the MERN-adjacent (MEN) stack. It establishes a solid, modular foundation — clear separation of routes, models, and controllers — that supports straightforward extension as new pages, an admin panel, and payment integration are added in future development cycles. The roadmap also includes security hardening — authentication, input validation, and rate limiting — to bring the platform closer to production readiness.